Data Wrangling & Exploration in Python

Using the GSS Subset Dataset

Victor Wandera Lumumba, MSc.
Statistician — Data Analyst — ML Expert

Mediacrest Training College

June 22, 2026

Learning Objectives

- Import and inspect real-world tabular data.
- Identify and handle missing values, duplicates, and outliers.
- Subset and filter rows based on logical conditions.
- Select specific columns or index ranges efficiently.
- Group data and compute aggregations.
- Engineer a new categorical income variable.
- Generate comprehensive descriptive statistics.

The Dataset: GSS Subset

- **File:** `GSSsubset.csv`
- **Records:** ∼3,000 respondents from the General Social Survey.
- **Columns:** `id`, `sex`, `degree`, `income`, `marital`, `age`, `height`, `weight`, `hrswrk`.

1. Data Importation

- CSV is the universal standard for tabular data.
- `read_csv()` handles parsing, type inference, and header alignment.
- Key parameters: `sep`, `header`, `na_values`, `dtype`.

```
import pandas as pd
import numpy as np

# Load the dataset
df = pd.read_csv('GSSsubset1.csv')
df.head()
```

2. Initial Data Inspection

- `.info()`: Reveals memory footprint, data types, and missing values.
- `.describe()`: Gives numeric summaries (mean, std, min, max, quartiles).
- `.value_counts()`: Shows category frequencies.

Diagnostic Checkpoint

This trio is your first diagnostic checkpoint before any cleaning begins.

Data Cleaning

3. Handling Missing Values

- Missing data appears as `NaN`, `None`, or placeholders.
- **Impact:** Skews statistics, breaks algorithms, reduces sample size.
- **Strategy:** Detect → Decide (Drop / Impute / Flag).

```
# Count missing per column
df.isna().sum()

# Percentage of missing data
df.isna().mean().round(4) * 100
```

Imputation Strategies

- **Numeric:** Use Median (safer for skewed data like income/weight).
- **Categorical:** Use Mode.

```
# Automatic numeric imputation
numeric_cols = df.select_dtypes(include='number').columns
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())

# Categorical imputation
cat_cols = df.select_dtypes(include=['object']).columns
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

4. Handling Duplicates

- **Exact duplicates:** Identical rows across all columns.
- **Impact:** Inflates counts, biases averages, breaks unique constraints.

```
# Count exact duplicate rows
dup_count = df.duplicated().sum()

# Remove duplicates (keeps first occurrence)
df_clean = df.drop_duplicates()
```

5. Detecting & Handling Outliers

- **Outliers:** Values far outside the expected range.
- **Detection:** IQR method ($Q1 - 1.5 \times IQR$, $Q3 + 1.5 \times IQR$).
- **Handling:** Cap/Winsorize to preserve sample size.

```
Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Cap outliers instead of removing rows
df[col] = np.where(df[col] < lower_bound, lower_bound, df[col])
df[col] = np.where(df[col] > upper_bound, upper_bound, df[col])
```

Data Manipulation

6. Filtering Rows

- Subset data based on logical conditions.
- Uses Boolean masking: `df[condition]`.
- Combine conditions with & (AND), | (OR), ~ (NOT).

```
# Single condition
males = df[df['sex'] == 'MALE']

# Multiple AND conditions
subset = df[(df['sex'] == 'FEMALE') &
            (df['age'] > 35) &
            (df['marital'] == 'MARRIED')]
```

Advanced Filtering

- `.query()`: Uses string expressions, highly readable.
- `.isin()`: Replaces long | chains for membership testing.

```
# Cleaner syntax with query()
df_filtered = df.query('age >= 25 and income > 50000')

# Filter using a list of values
degrees = ['BACHELOR', 'GRADUATE', 'JUNIOR COLLEGE']
df_edu = df[df['degree'].isin(degrees)]
```

7. Selecting Columns

- Extract specific variables for focused analysis.
- Methods: bracket notation, `.loc`, `.iloc`.

```
# Multiple columns selection
cols = ['sex', 'age', 'income', 'degree']
df_subset = df[cols]

# Label-based selection
df_labels = df.loc[:, 'age':'weight']

# Filter by data type
numeric_cols = df.select_dtypes(include='number').columns
```

8. Grouping By (Split-Apply-Combine)

- Split data by category, apply function, combine results.
- Essential for comparative analysis (e.g., income by degree).

```
# Mean income by education level
edu_income = df.groupby('degree')['income'].mean()

# Group by multiple columns
grouped = df.groupby(['sex', 'marital'])['income'].mean()
```

Advanced Grouping

- `.agg()` lets you run multiple statistics simultaneously.
- Flattening column names prevents `KeyError` issues downstream.

```
summary = df.groupby('degree').agg({
    'income': ['mean', 'median', 'std', 'count'],
    'age': 'mean',
    'hrswrk': 'sum'
})

# Flatten multi-level column names
summary.columns = ['_'.join(col) for col in summary.columns]
```

Feature Engineering & Statistics

9. Feature Engineering

- Create new variables from existing ones.
- Convert continuous income to categorical `Income_Level`.

```
median_income = df['income'].median()

# Method 1: np.where (fast, vectorized)
df['Income_Level'] = np.where(
    df['income'] >= median_income,
    'High Income',
    'Low Income'
)

# Method 2: pd.cut (handles edges cleanly)
bins = [0, median_income, np.inf]
df['Income_Level_cut'] = pd.cut(df['income'], bins=bins,
                                labels=['Low', 'High'])
```

10. Descriptive Statistics

- **Numeric:** mean, median, std, skewness, kurtosis.
- **Categorical:** frequencies, mode, unique counts.

```
# Numeric summary
df.describe()

# Categorical summary
df.describe(include=['object', 'category'])

# Distribution shape metrics
df['income'].skew()
df['income'].kurtosis()
```

11. Correlation Analysis

- Examine linear relationships between continuous variables.

```
numeric_vars = ['age', 'height', 'weight', 'hrswrk', 'income']
corr_matrix = df[numeric_vars].corr()
print(corr_matrix.round(3))
```

Important

Correlation $\neq$ Causation! Always consider context and confounding variables.

12. Exporting Cleaned Data

- Always save your cleaned/processed data for reproducibility.
- Document your decisions for auditability.

```
# Create a final cleaned dataset
df_export = df.copy()

# Add derived variables (e.g., BMI)
df_export['BMI'] = (df_export['weight'] * 0.453592) / \
                   ((df_export['height'] * 0.0254) ** 2)

# Save to CSV
df_export.to_csv('GSSsubset_processed.csv', index=False)
```

Summary & Key Takeaways

- Proper import and inspection prevent downstream errors.
- Handle missing values, duplicates, and outliers systematically.
- Use boolean masking and `.query()` for efficient filtering.
- Grouping and feature engineering unlock deeper insights.
- Always export and document your cleaned data!

Q&A and Thank You!

Questions?

Victor Wandera Lumumba
Statistician — Data Analyst — ML Expert

Email: lumumbavictor172@gmail.com
Web: beyonddataanalytics.online
GitHub: Lumumba1992